Gerenciamento de Perfis Distribuídos Através de Inferência de Trilhas

André Wagner
Jorge Barbosa

22 de Setembro de 2011

1 Introdução

Atualmente, aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de permitir que as aplicações possam conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em *perfis de usuário*, onde cada aplicação salva as informações em um formato baseado em um *modelo de usuário* específico.[1]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[2] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade é uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o *perfil* torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

A capacidade de uma aplicação perceber o contexto de uma entidade pode ser amplificada através do uso de *trilhas*[3]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade dentro do contexto.

A coleta de informações de um usuário pode ser *explícita* ou *implícita*[1]. Na coleta *explícita*, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta *implícita*, a informação é inferida a partir do estudo do comportamento dos usuários.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [1]. No entanto, os diferentes *modelos de usuário* utilizados pelas aplicações tornam difícil a utilização de informações de um *perfil de usuário* de uma aplicação por outra. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um *contexto* específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Como forma de ultrapassar estas barreiras, o conceito de *personalização inter-sistemas* foi introduzido [4] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de *sistemas de modelos de usuário* é o principal desafio da pesquisa atual na área de personalização.[1]

O IEEE Standard Computer Dictionary [5] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a "capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogenias e integrar esta informação em um modelo de usuário de granularidade própria" [6]. Os dois principais modelos de interoperabilidade são: (i) *interoperabilidade sintática*, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) *interoperabilidade semântica*, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [1]:

Baseados em padrões : são baseados numa visão *top-down*, definindo um padrão que todas

as aplicações envolvidas devem observar;

Baseados em mediação : se baseiam numa visão *bottom-up*, onde ocorre algum tipo de *reconciliação* entre os diferentes modelos de representação.

2 Trabalhos relacionados

Esta seção apresenta projetos relacionados a este, bem como uma comparação entre os mesmos.

2.1 PeLeP

O projeto PeLeP, apresentado por Levis et al[7], busca integrar a utilização de perfis com aperfeiçoamento automático com ambientes de educação ubíqua.

Os ambientes de *computação ubíqua* são parte de um cenário na qual serviços e informações estão disponíveis a qualquer momento, em qualquer lugar[8]. Esta visão vem sendo potencializada pela diminuição do custo e aumento da capacidade de memória e processamento dos dispositivos computacionais, bem como pela cada vez maior disponibilidade de recursos de localização e conectividade. A computação ubíqua enfatiza também a "tecnologia calma", onde a sobrecarga de informação é substituída pela disponibilização de recursos de forma inteligente, tomando a atenção do usuário apenas nos momentos em que é mais relevante.[9]

Levis et al compreendem *educação ubíqua* como sendo processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz. Desta forma, o conhecimento presente no dia-a-dia das mais diferentes formas e em diferentes locais podem ser relacionados com processos educacionais direcionados aos aprendizes.

O PeLeP, cujo modelo é ilustrado na Figura 1, é um modelo dedicado ao gerenciamento de perfis de aprendizes em um ambiente ubíquo de ensino e aprendizagem. O modelo administra os perfis automaticamente, inferindo informações através do histórico do aprendiz. As alterações realizadas no perfil refletem o comportamento do aprendiz no ambiente ubíquo, levando em consideração o seu modelo de mobilidade e de contexto.

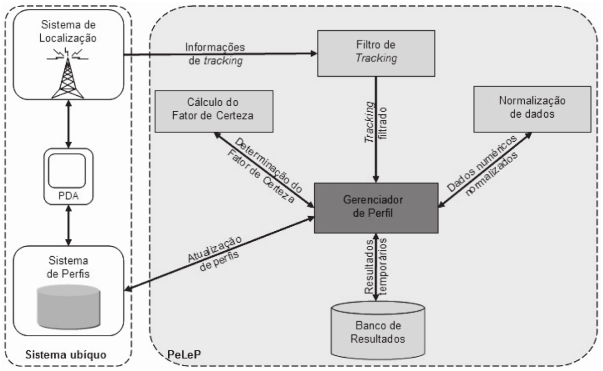

Figura 1: Arquitetura do PeLeP[7]

2.2 Chellouche et al

O artigo de Chellouche et al[10] discute as dificuldades atuais no desenvolvimento de aplicativos para dispositivos móveis, uma vez que os mesmos possuem uma grande variedade de resolução, memória, poder computacional, etc. Além disso, a qualidade da conexão de um usuário pode variar grandemente, e isto também deve ser levado em consideração no desenvolvimento.

O conjunto de informações que caracteriza um usuário é chamada de *perfil de usuário*. Com a intenção de compartilhar o contexto do usuário entre serviços e minimizar as mudanças no desenvolvimento de aplicações sensíveis a contexto, os autores propõem como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O perfil do usuário é constituído por seu contexto, onde a definição de contexto utilizada é aquela dada por Dey[2], na qual contexto refere-se a qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade. Assim, o perfil de usuário é dividido em cinco componentes:

Perfil geral : refere-se às informações básicas, como nome, endereço, telefone, idade, sexo, etc. Inclui também possíveis deficiências pessoas do usuário.

Perfil do dispositivo : informações a respeito dos dispositivos do usuário, tais como marca, capacidade, sistema operacional, etc.

Perfil de rede : descreve as redes às quais o usuá-

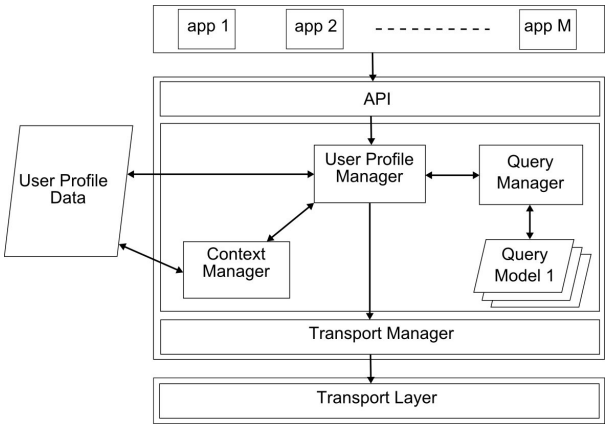

Figura 2: Arquitetura do *middleware* de Chellouche et al[10]

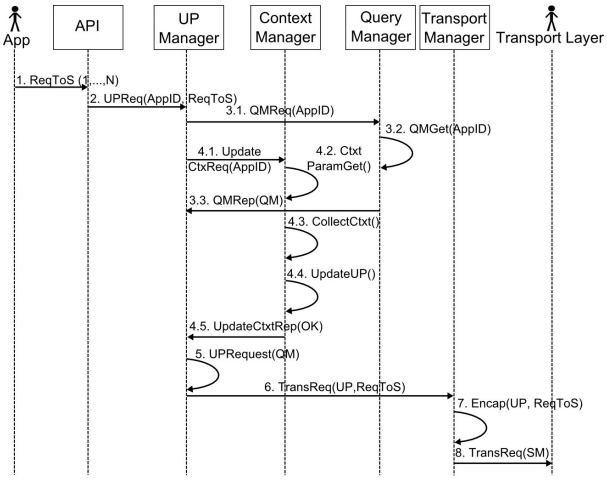

Figura 3: Construção de perfil[10]

rio pode se conectar, com suas características.

Perfil dos serviços : registra as informações a respeito de serviços, como nome, versão, porta e conteúdo.

Perfil de contexto : contém informações a respeito do ambiente do usuário, como hora, data e localização, aplicações em execução, banda disponível, etc. É considerada a parte dinâmica do perfil, composta por dados voláteis.

O *middleware* é apresentado segundo ilustrado na Figura 2. O *middleware* é composto dos seguintes módulos:

User profile manager : módulo central da arquitetura, que interage com todos os outros módulos com o objetivo de construir o perfil correto.

Context manager : monitora o ambiente, coletando informações com o objetivo de atualizar a parte dinâmica do perfil.

Query manager : módulo responsável por validar e salvar um modelo de consulta para cada aplicação. É construída quando a aplicação é instalada e utilizada quando a aplicação solicita informações.

Transport manager : faz a comunicação do perfil de usuário para a aplicação.

API : constitui o ponto de interface entre as aplicações e o middleware.

O perfil é construído de acordo com a Figura 3. A aplicação solicita ao gerenciador de perfis, através da API, que por sua vez repassa a solicitação ao gerenciador de consultas. Simultâneo a isto, o gerenciador de contextos é informado que a aplicação está em execução e passa a atualizar o gerenciador de perfis com as informações atualizadas. O perfil de usuário é criado e enviado ao gerenciador de transporte, que retorna o perfil solicitado à aplicação solicitante.

2.3 Outros

[Avaliação de outros trabalhos relacionados]

2.4 Avaliação dos Ambientes Pesquisados

A Tabela 1 exibe um comparativo dos trabalhos apresentados, onde os seguintes fatores são levados em consideração:

Interoperabilidade : o modelo apresenta interoperabilidade – caso positivo, se a mesma é sintática ou semântica;

Perfis dinâmicos : o modelo de perfil permite que os perfis sejam atualizados de acordo com eventos externos, sem a interferência do usuário;

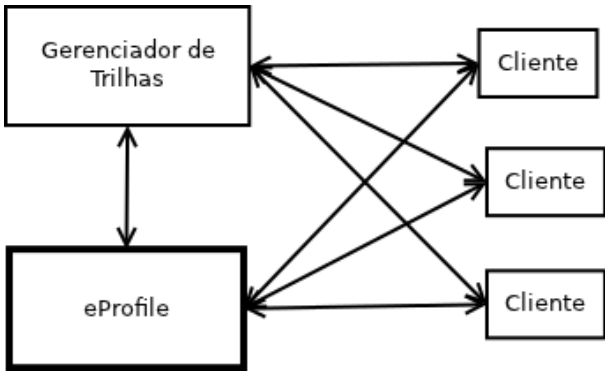

Figura 4: Modelo do eProfile

Armazenamento : define onde ficam armazenados os perfis do usuário;

Sensível a contexto : o contexto do usuário é levado em consideração na formação do perfil;

Utiliza trilhas : as trilhas do usuário são levadas em consideração na construção do perfil;

Domínio : o perfil é voltado para um domínio específico, ou se o mesmo permite que vários domínios possam ser utilizados.

O modelo apresentado por Chellouche et al[10] apresenta um *middleware* que pode ser usado por diferentes aplicativos como um servidor central de perfis. Ele é voltado para aplicações móveis e, embora atualize parte do perfil como forma de oferecer suporte a sensibilidade a contexto, não permite que os aplicativos criem novos campos nem definam as regras para atualização dos campos existentes.

3 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através da análise das trilhas de uma entidade (disponibilizadas por um gerenciador de trilhas), seguindo regras definidas pelas próprias entidades, e disponibilizado os dados inferidos para serem utilizados por múltiplas aplicações.

A Figura 4 apresenta o modelo, que é organizado em três partes.

O eProfile é o modelo apresentado neste artigo. Ele interage com o gerenciador de trilhas através de uma interface padronizada, de onde busca as informações necessárias para a inferência dos perfis.

Para que o eProfile possa operar, é necessário que o mesmo esteja conectado a um gerenciador de trilhas que implemente uma interface padrão. O gerenciador de trilhas é o módulo responsável por armazenar as trilhas, fornecendo uma interface que permita a consulta de um conjunto destas trilhas dentro de parâmetros específicos (por exemplo, entidade ou data). O gerenciador de trilhas deve fornecer as trilhas no formato especificado pelo eProfile.

Os clientes referem-se a dispositivos que se comunicam com o gerenciador de trilhas e com o eProfile. Estes clientes fazem o registro de seus eventos dentro do gerenciador de trilhas. O eProfile e o gerenciador de trilhas trabalham em conjunto, independente dos clientes, de modo a manter as informações de perfil pré-processadas dentro do eProfile.

Cada cliente é responsável por fornecer o conjunto de regras para o processamento de suas próprias trilhas. Desta forma, cada cliente pode registrar os eventos de histórico no formato que considerar pertinente, desde que as regras respeitem o formato utilizado.

3.1 Modelagem

O eProfile é um sistema orientado a agentes, onde a metodologia utilizada para a modelagem é a Prometheus[11]. Esta metodologia define um processo detalhado para a especificação, projeto, implementação e teste/depuração de sistemas de software orientados a agentes.[12]

A metodologia Prometheus, ilustrada na Figura 5 define que o primeiro passo da modelagem de um sistema é a identificação dos objetivos inciais do sistema. De acordo com o modelo do eProfile, os objetivos do sistema são:

- Gerenciar múltiplos perfis;
- Comunicar-se com um (ou mais) gerenciadores de trilhas;
- Receber as regras de processamento dos clientes;
- Inferir dados de perfil através da análise das trilhas, de acordo com as regras definidas pelos clientes;

Trabalho	Interope-rabilidade	Perfis dinâmicos	Armaze-namento	Sensibilidade a contexto	Utiliza trilhas	Domínio
Chellouche et al [10]	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia

Tabela 1: Comparação entre os trabalhos apresentados

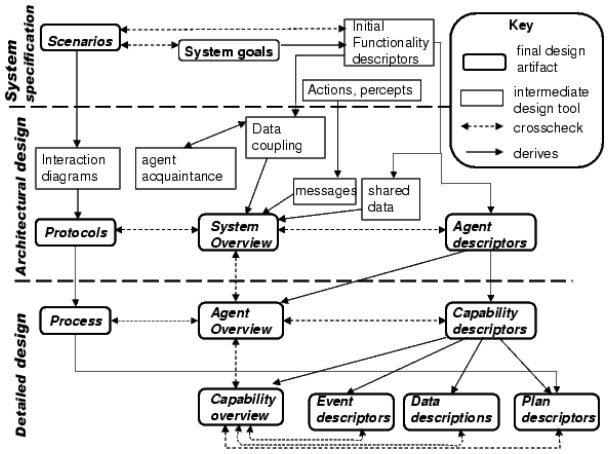

Figura 5: Processo de projeto segundo a metodologia Prometheus [12]

- Aceitar requisições de informações de perfil pelos clientes.

Referências

[1] M. Viviani, N. Bennani, and E. Egyed-Zsigmond, "A Survey on User Modeling in Multi-Application Environments," in *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pp. 111–116, IEEE, Aug. 2010.

[2] A. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 6, no. 1, pp. 4–7, 2001.

[3] J. Silva, J. a. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo, "Content distribution in trial-aware environments," in *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, (New York, NY, USA), pp. 15:1–15:8, ACM, 2009.

[4] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje, "A Multi-Dimensional, Unified User Model for Cross-System Personalization," in *E4PIA Workshop 2004*, 2004.

[5] "Ieee standard computer dictionary. a compilation of ieee standard computer glossaries," *IEEE Std 610*, p. 1, 1991.

[6] F. Carmagnola and F. Cena, "User identification for cross-system personalisation," *Inf. Sci.*, vol. 179, pp. 16–32, January 2009.

[7] D. Levis, J. L. V. Barbosa, and S. C. C. S. Pinto, "Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua," *Revista Brasileira de Informática na Educação*, vol. 16, no. 1, 2008.

[8] M. Weiser, "The computer for the 21st century," *Scientific America*, pp. 94–104, 1991.

[9] M. Weiser and J. S. Brown, "Designing calm technology," *PowerGrid Journal*, July 1996.

[10] S. A. Chellouche, J. Arnaud, and D. Négru, "Flexible user profile management for context-aware ubiquitous environments," in *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, (Piscataway, NJ, USA), pp. 980–984, IEEE Press, 2010.

[11] L. Padgham and M. Winikoff, "Prometheus: A Methodology for Developing Intelligent Agents," 2002.

[12] L. Padgham and M. Winikoff, *Developing intelligent agent systems: a pratical guide.* John Wiley and Sons, Ltd, 2004.